DALHOUSIE UNIVERSITY

# Reproducible Reporting

R Markdown and Python notebooks

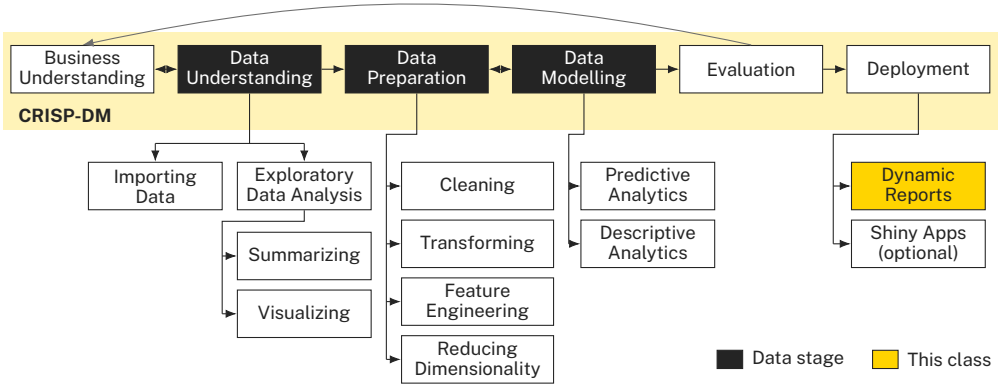
**Prof. Gabriel Spadon, PhD**
Faculty of Computer Science
spadon@dal.ca

# Turn an Analysis into a Reproducible Report

- Combine a question, code, evidence and interpretation.
- Produce a PDF from a fresh R process or Python kernel.
- Distinguish execution errors from conversion errors.
- Record the inputs, parameters and environment.

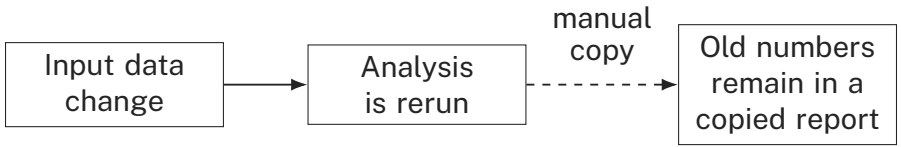
Choose R Markdown or a Python notebook. The complete examples in the notes support either route.

# Communicate the Analysis to Its Reader




Dynamic reports communicate earlier work; Shiny apps are an optional sibling.

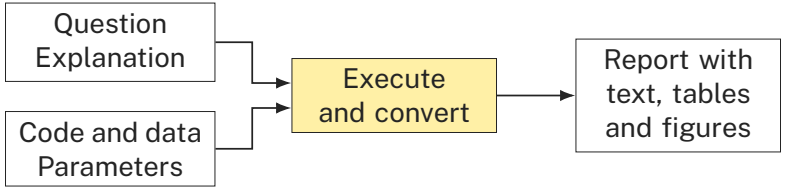
# A Copied Result Can Become Stale




Copied numbers can remain stale after the analysis changes.
Run the linked report again and inspect the output.

Computed text can update a count after a rerun. An ordinary interpretation sentence still needs to be checked.

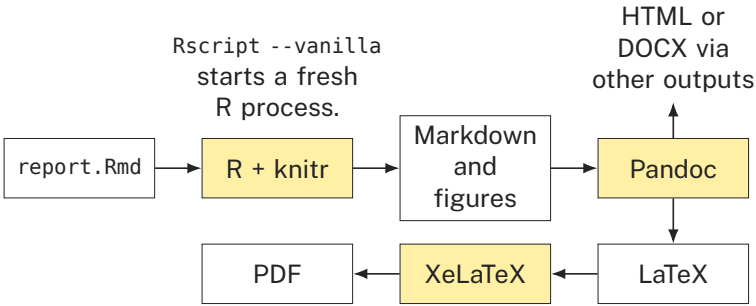
# Connect Explanation and Computation




A source links the analysis to the report.
Run it again to update the output.

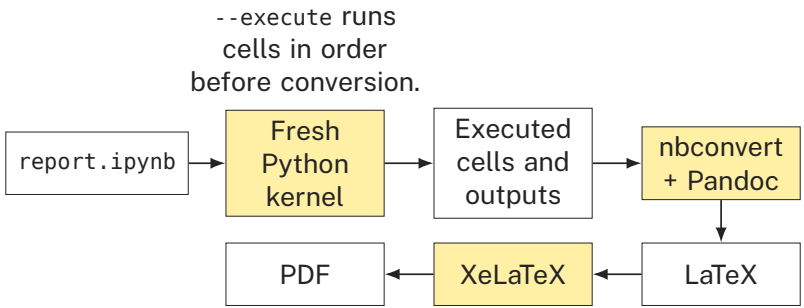
A PDF records one run. The source still needs its input data and software to reproduce that run.

# Separate Execution from Conversion




R and knitr execute enabled chunks. Pandoc converts the document; XeLaTeX compiles the PDF.

# Execute Cells Before Exporting the Report




An ipynb file stores cells, metadata and possibly old outputs. Exporting those saved outputs alone does not recompute the analysis.

# Keep the Source and Inputs Together

For Lab 04, work in your own repository's `labs/lab-04` folder.

- Save `report.Rmd` or `report.ipynb`.
- Keep input paths relative to that working folder.
- Retain raw data and record preparation choices.
- Run the command in the folder that contains the report.

Use a separate practice folder for each class example. The notes contain all source text and the full CSV.

# Prepare One Language Environment

DALHOUSIE UNIVERSITY

**R Markdown**

R and an editor such as RStudio.

R packages `rmarkdown`, `knitr`, `ggplot2`, `dplyr`.

Pandoc and XeLaTeX for the PDF route.

**Python notebook**

JupyterLab and a Python kernel.

pandas, NumPy, matplotlib, scikit-learn, IPython, nbconvert and Jinja2.

Pandoc and XeLaTeX for the PDF route.

Install dependencies once in the chosen environment. The notes give the setup and kernel-selection commands.

# Save Files Beyond the Runtime

- Local RStudio or Jupyter keeps files on your computer.
- Posit Cloud provides projects that can be downloaded.
- Colab saves notebooks separately from its temporary runtime files.
- For Kaggle or another supported option, check language, persistence and export before starting.

If cloud PDF tools are unavailable, download the source and inputs and render locally. Keep coursework private until the deadline and instructor release.

# Use the Familiar Iris Measurements

Iris contains 150 flowers, 50 of each species, measured in centimetres.

- One compact table compares species means.
- A histogram shows the pooled sepal-length distribution.
- Box plots compare variation within and between species.

The R and scikit-learn copies agree.

# Start with Document Settings

```
---
title: "Iris sepal lengths"
author: "Course example"
date: ""
output:
  pdf_document:
    latex_engine: xelatex
    fig_caption: true
  html_document: default
---
```

The YAML header names the output formats. Its first format is the default; a render command can select another.

# Run Setup Without Printing It

````
```{r setup, include=FALSE}
library(ggplot2)
library(dplyr)
knitr::opts_chunk$set(
  echo = FALSE, cache = FALSE, error = FALSE,
  warning = TRUE, message = FALSE,
  fig.width = 6, fig.height = 3, out.width = "90%"
)
dat <- iris
```
````

The setup executes. `include=FALSE` hides its code and output, while `cache=FALSE` keeps this small analysis fresh.

# Put a Calculation into an R Sentence

```
## Question and data

How do sepal lengths vary within and between the three species in this
sample? Iris contains 150 flowers, 50 of each species, with measurements
in centimetres.

The mean sepal length is `r sprintf("%.2f", mean(dat$Sepal.Length))` cm,
with sample standard deviation `r sprintf("%.3f", sd(dat$Sepal.Length))` cm.

## A compact summary
```

The rendered mean is 5.84 cm and sample standard deviation is 0.828 cm. These values update when this source is rerendered.

# Emit the Same Sentence from Python

Run the Iris Python setup cell in the notes before this example.

```
mean_sepal = dat["Sepal.Length"].mean()
sd_sepal = dat["Sepal.Length"].std(ddof=1)
display(
    Markdown(
        f"The mean sepal length is {mean_sepal:.2f} cm, "
        f"with sample standard deviation {sd_sepal:.3f} cm."
    )
)
```

A normal Markdown cell does not evaluate Python. This code cell formats its result as Markdown output; ddof=1 matches R's sample standard deviation.

# Compute a Compact Group Summary

## R

```r
```{r species-summary}
summary_table <- dat |>
  group_by(Species) |>
  summarise(
    n = n(), Sepal_cm = mean(Sepal.Length),
    Petal_cm = mean(Petal.Length), .groups = "drop"
  )
knitr::kable(summary_table, row.names = FALSE, digits = 3)
```
```

## Python

```python
summary_table = (
    dat.groupby("Species", sort=True)
    .agg(
        n=("Sepal.Length", "size"),
        Sepal_cm=("Sepal.Length", "mean"),
        Petal_cm=("Petal.Length", "mean"),
    )
    .reset_index()
)
```

Both give one row per species with a count and two means. In R, kable formats the table; Python keeps Species as a named column.

# Keep Counts and Units Visible

| Species | $n$ | Sepal mean (cm) | Petal mean (cm) |
|---|---|---|---|
| setosa | 50 | 5.006 | 1.462 |
| versicolor | 50 | 5.936 | 4.260 |
| virginica | 50 | 6.588 | 5.552 |

Every species has 50 observations. Means alone do not reveal the spread, so the report also needs plots.

# Supply the Table to Both Export Formats

```
display(
    {
        "text/html": summary_table.to_html(
            index=False, float_format="%.3f", escape=True
        ),
        "text/latex": summary_table.to_latex(
            index=False, float_format="%.3f", escape=True
        ),
    },
    raw=True,
)
```

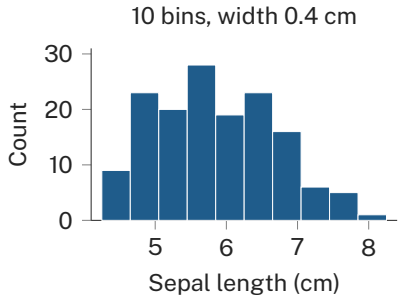
HTML and LaTeX are two representations of the same calculation. Each exporter selects the one it can use.

# Record the Bin Grid in the Report

````
```{r histogram, fig.cap="Sepal length distribution."}
edges <- seq(4.25, 8.25, length.out = 11)
ggplot(dat, aes(Sepal.Length)) +
  geom_histogram(
    breaks = edges, closed = "left",
    fill = "#6E6E6E", colour = "white"
  ) +
  labs(x = "Sepal length (cm)", y = "Count") +
  theme_minimal(base_size = 11)
```
````

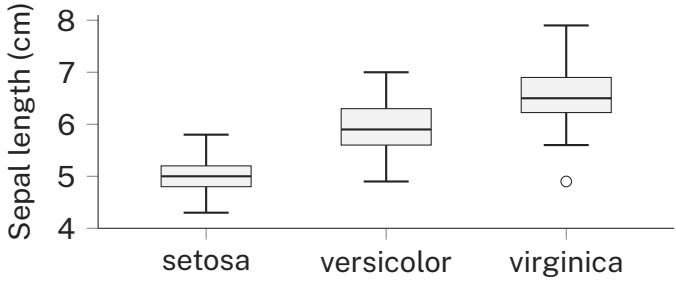
The same Class 06 grid is used in Python. The ten counts sum to 150; none of the measurements lies on an internal edge.

# Read the Pooled Distribution




Bins have width 0.4 cm and run from 4.25 to 8.25 cm. The plot combines all three species.

# Compare the Species with Box Plots




Both report routes use linear quartiles and 1.5-IQR whiskers, as in Class 06. The separate virginica point is 4.9 cm; its lower whisker is 5.6 cm.

# Distinguish Device Size from Display Size

DALHOUSIE UNIVERSITY

## R

`fig.width` and `fig.height` are in inches.

`fig.asp` is a height-to-width ratio.

`out.width` controls display width in the report.

## Python

`figsize=(6,3)` sets a six-by-three inch figure.

Label the axes, apply `tight_layout()`, then check the exported size.

Shrinking a figure after drawing it also shrinks the labels. A caption should explain the units and plotting convention.

# Render from a Fresh Process or Kernel

```
Rscript --vanilla -e "rmarkdown::render(
  'report.Rmd', output_format='pdf_document')"
```

```
jupyter nbconvert --execute --to pdf --no-input report.ipynb
```

Run the command for your language in the report folder. Both produce `report.pdf`. The notebook flag `--no-input` hides code; `--execute` still runs it.

# Check the Result That Will Be Submitted

1. Confirm the command completed without an execution error.
2. Open the new PDF and inspect every page.
3. Verify one count and calculated value against the input.
4. Check every plot, unit, table and interpretation.
5. Preserve the source, input identity and software versions.

A successful render proves this configured run completed. It does not prove that the analysis or conclusion is correct.

# Separate Running Code from Showing Code

| Option | R Markdown default | Effect |
|---|---|---|
| `eval` | `TRUE` | Run the code |
| `echo` | `TRUE` | Show source code |
| `include` | `TRUE` | Include source and output |
| `results` | `'markup'` | Format text output |
| `message` | `TRUE` | Include messages |
| `warning` | `TRUE` | Include warnings |
| `error` | `FALSE` | Stop on a code error |
| `cache` | `FALSE` | Reuse cached results if enabled |

Give each chunk a unique label such as `species-summary`. Required analysis must still run.

# Use the Settings of the Selected Output

`fig.cap` supplies an R figure caption, and the example's PDF settings enable numbered figure captions.

- Numbering and float placement depend on the output format.
- Python examples put a numbered title inside each figure.
- Keep captions beside their figures in the final PDF.
- HTML is useful for checking, but Lab 04 requires PDF.

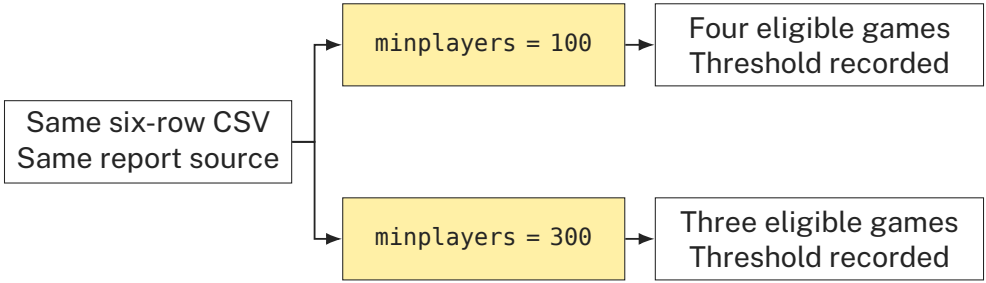
The notes also explain results, message and error options.

# Saved Output Can Come from an Earlier Run

- R notebooks support interactive chunk output and HTML previews.
- Python notebooks can retain outputs from cells run out of order.
- An output-format name does not itself start a fresh session.
- Rebuild with the tested process or kernel before final export.

An ipynb is JSON text. Its outputs and metadata can make diffs noisy, but its source and history still belong in the reproducible record.

# Name an Input Choice and Record It




A parameter changes the selection while preserving one analytical recipe. The report should state the value used.

# Use Six Game Records

```
game,players,completed,achievements
Celeste,412,254,32
Hades,388,180,49
Stardew,301,112,40
Factorio,154,39,27
Spelunky,88,21,25
Tunic,64,12,26
```

This is a separate game-level dataset, not an aggregation of Class A's play records. Save the CSV beside the report.

# Define the Completion Rate First

$$\text{completion rate} = \frac{\text{completed players}}{\text{listed players}}$$

- A row represents one game.
- Completed players cannot exceed listed players.
- Players must be positive for a defined rate.
- Achievement count is not the rate denominator.

The source stops on a missing value, a nonpositive player count or more completions than players, before creating a filtered copy.

# Declare the Default Values

## R

```
---
title: "Completion rates by game"
author: "Course example"
date: ""
output:
  pdf_document:
    latex_engine: xelatex
    fig_caption: true
  html_document: default
params:
  datafile: "steam_achievements.csv"
  minplayers: 100
---
```

## Python

```
datafile = "steam_achievements.csv"
minplayers = 100
```

A plain cell near the top holds the defaults. The preparation cell after it validates the counts, filters a copy of `raw` and computes the rate.

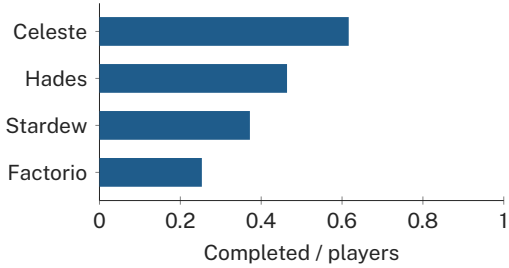
Both declare the same two defaults. The datafile names a sibling CSV with this schema; Iris would not fit it.

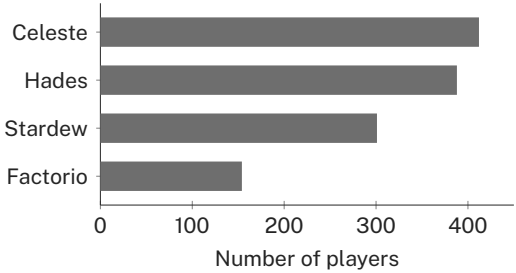
# Verify the Default Table

| Game | Players | Completed | Achievements | Rate |
|---|---|---|---|---|
| Celeste | 412 | 254 | 32 | 0.617 |
| Hades | 388 | 180 | 49 | 0.464 |
| Stardew | 301 | 112 | 40 | 0.372 |
| Factorio | 154 | 39 | 27 | 0.253 |

At least 100 players retains four of six games. Completed divided by players gives each rate; the achievement count remains a separate quantity.

# Show Rates and Their Denominators




Keep games in the same order. The rate plot compares proportions; the count plot reveals how many players each proportion describes.

# Preserve a Distinct Output for Each Choice

```
Rscript --vanilla -e "rmarkdown::render(
  'report.Rmd', params=list(minplayers=300),
  output_file='report-300.pdf')"
```

```
jupyter nbconvert --execute --to pdf --no-input --output report-300 report.ipynb
```

For Python, first change and save the parameter cell to 300. Both versions retain three games. Their per-game rates stay the same.

# Report an Empty Selection Honestly

With `minplayers = 1000`, none of the six games is eligible.

- State that no games met the threshold.
- Do not present an empty chart as a meaningful comparison.
- Preserve the chosen threshold in the report.

# Read the First Informative Error

- A missing name or package points to setup and the selected environment.
- A missing file points to the working folder and relative path.
- A Pandoc error points to document conversion.
- A XeLaTeX error points to the PDF compilation log.

A working HTML export can help isolate a PDF-stage problem. It does not prove that the final PDF is correct or that every required computation ran.

# Preserve More Than the Final PDF

- Source, raw input identity, parameters and software versions.
- Units, exclusions, cleaning decisions and analytical limits.
- The PDF from the final fresh run.
- Meaningful commits and the submitted full commit SHA.

Preserve existing history. Do not amend, squash, rebase, force-push or backdate submitted work. Publish only after the deadline and instructor release.

# Cite the Official Data Record in the Report

A short Markdown section near the end is enough; no bibliography file is needed.

```
## Data source

Nova Scotia Department of Energy. *Low Carbon Communities
Grant Recipients* (record p9y5-d99g). Nova Scotia Open
Government Licence. https://data.novascotia.ca/d/p9y5-d99g,
accessed [YYYY-MM-DD].
```

Take the details from the catalogue page and replace the placeholder with your download date. Refer to the section where the data are first described.

# From One Source to a Checked PDF

DALHOUSIE UNIVERSITY

- One source rebuilds text, tables and figures; check the rendered PDF.
- Execution, conversion and TeX fail separately; the stage names the log.
- Render from a fresh R process or with `--execute`.
- `eval` controls running; `echo` and `include` control showing.
- A parameter changes coverage, not per-row values; state its value.
- Report an empty selection plainly rather than draw an empty chart.
- A clean render shows the source made the PDF, not that it is right.

# Build, Change and Explain One Report

1. Choose R or Python and build the Iris report.
2. Export a fresh PDF and verify its count, mean and plots.
3. Change the game threshold from 100 to 300.
4. Explain the coverage change and distinguish rate from count.